

## Java 19 — Virtual Threads (Preview)

**Java Version:** 19

**Year:** 2022

**Feature:** Virtual Threads — Preview

**UniqueClassName:** VirtualThreadsPreview

### Definition

**Virtual Threads** were introduced as a **preview feature in Java 19** as part of **Project Loom**.

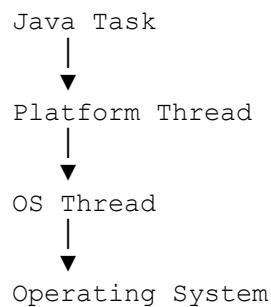
A virtual thread is a lightweight thread managed by the **JVM**, rather than being directly tied to a traditional operating-system thread.

The main goal is to make it practical to run **very large numbers of concurrent tasks**, particularly tasks that spend significant time waiting for I/O such as:

- HTTP requests
- Database operations
- File operations
- Network communication

### Traditional Platform Threads

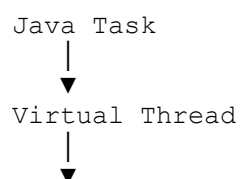
With traditional Java threads:

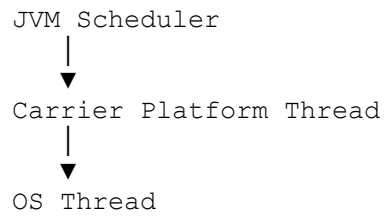


Creating thousands or millions of platform threads can consume substantial memory and OS resources.

### Virtual Threads

With virtual threads:

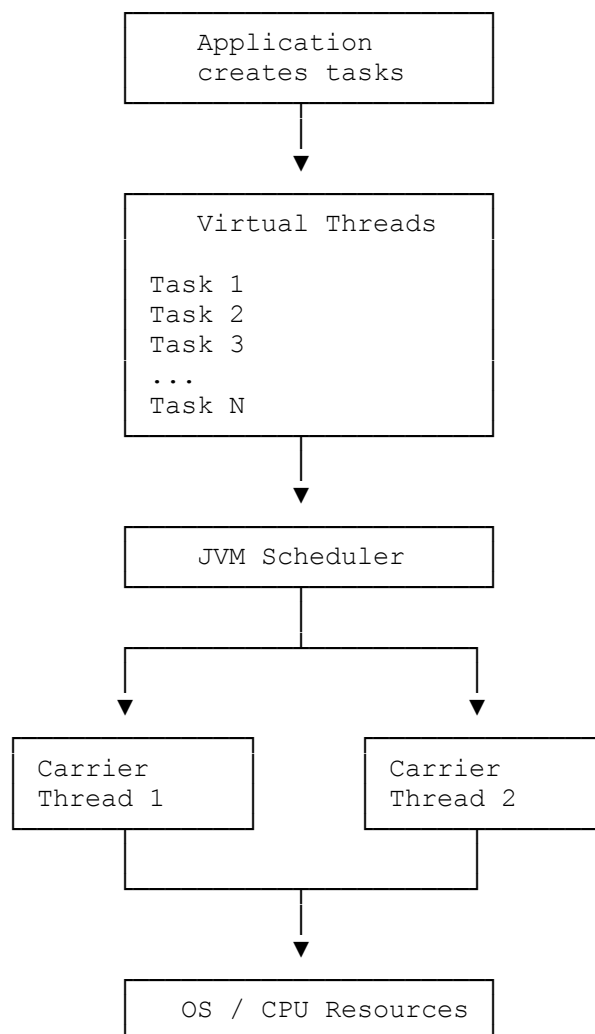




Many virtual threads can share a much smaller number of platform/carrier threads.

---

## Flowchart



## Java Code Example

Java 19 requires preview features to be enabled when compiling and running this example.

```
public class VirtualThreadsPreview {  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Thread virtualThread = Thread.startVirtualThread(() -> {  
  
            System.out.println(  
                "Running on: "  
                + Thread.currentThread()  
            );  
  
            try {  
                Thread.sleep(1000);  
            } catch (InterruptedException e) {  
                Thread.currentThread().interrupt();  
            }  
  
            System.out.println("Task completed");  
        });  
  
        virtualThread.join();  
    }  
}
```

## Compile

Because Virtual Threads were preview functionality in Java 19:

```
javac --enable-preview --release 19 VirtualThreadsPreview.java
```

## Run

```
java --enable-preview VirtualThreadsPreview
```

Possible output:

```
Running on: VirtualThread[#21]/runnable@ForkJoinPool-1-worker-1  
Task completed
```

---

## Creating Many Virtual Threads

One of the major advantages is that you can create a very large number of virtual threads.

```

public class VirtualThreadsPreview {

    public static void main(String[] args)
        throws InterruptedException {

        Thread[] threads = new Thread[10_000];

        for (int i = 0; i < threads.length; i++) {

            int taskNumber = i;

            threads[i] = Thread.startVirtualThread(() -> {

                System.out.println(
                    "Task " + taskNumber
                    + " running on "
                    + Thread.currentThread()
                );
            });
        }

        // Wait for all virtual threads
        for (Thread thread : threads) {
            thread.join();
        }

        System.out.println("All tasks completed.");
    }
}

```

Compile:

```
javac --enable-preview --release 19 VirtualThreadsPreview.java
```

Run:

```
java --enable-preview VirtualThreadsPreview
```

The important idea is that **10,000 concurrent tasks do not require 10,000 dedicated OS threads.**

---

## Using Thread.Builder

Java 19 also introduced APIs for creating virtual threads through the Thread.Builder API.

```

public class VirtualThreadsPreview {

    public static void main(String[] args)
        throws InterruptedException {

```

```

        Thread thread = Thread.ofVirtual()
            .name("virtual-worker")
            .start(() -> {

                System.out.println(
                    "Thread name: "
                    + Thread.currentThread().getName()
                );
            });

        thread.join();
    }
}

```

Output:

Thread name: virtual-worker

---

## Platform Thread vs Virtual Thread

```

Platform Thread
    |
    ▼
OS Thread
    |
    ▼
Higher resource consumption
    |
    ▼
Limited scalability

```

Compared with:

```

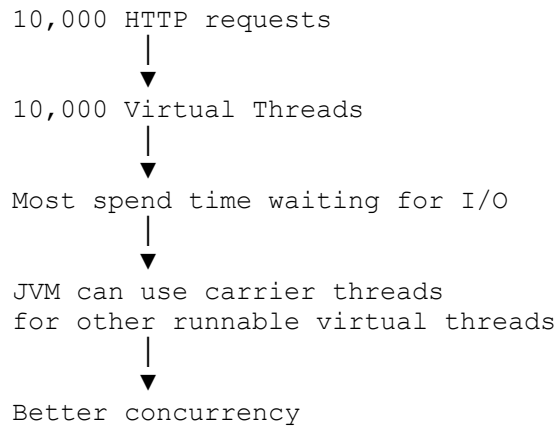
Virtual Thread
    |
    ▼
JVM-managed
    |
    ▼
Carrier Thread
    |
    ▼
OS Thread
    |
    ▼
Very high concurrency

```

## Important Point

Virtual threads are especially useful for **I/O-bound workloads**.

For example:



They do **not** make CPU-intensive calculations inherently faster. If the workload is CPU-bound, the available CPU cores remain the fundamental constraint.

---

## Key Characteristics

| Characteristic               | Virtual Threads        |
|------------------------------|------------------------|
| Introduced                   | Java 19                |
| Year                         | 2022                   |
| Status in Java 19            | Preview                |
| Project                      | Project Loom           |
| JVM-managed                  | Yes                    |
| OS thread per virtual thread | No                     |
| Lightweight                  | Yes                    |
| High concurrency             | Yes                    |
| Particularly useful for      | I/O-bound applications |
| Thread.startVirtualThread()  | Yes                    |

|                      |         |
|----------------------|---------|
|                      |         |
| <b>Final feature</b> | Java 21 |

## In short

**Java 19 introduced Virtual Threads as a preview feature, allowing applications to create huge numbers of lightweight JVM-managed threads. They are designed primarily to simplify and scale concurrent, I/O-heavy applications while using far fewer underlying platform threads.**

## Java 19 — Record Patterns (Preview)

**Java Version:** 19

**Year:** 2022

**Feature:** Record Patterns — Preview

**UniqueClassName:** RecordPatternsPreview

## Definition

**Record Patterns** were introduced as a **preview feature in Java 19**.

They allow you to **deconstruct a record object and extract its components directly during pattern matching**.

Record patterns work especially well with the **Pattern Matching for instanceof** feature.

## Without Record Pattern

Suppose we have:

```
record Person(String name, int age) {}
```

Traditionally, you would first check the type and then call the record accessors:

```
if (obj instanceof Person person) {
    String name = person.name();
    int age = person.age();

    System.out.println(name);
    System.out.println(age);
}
```

## With Record Pattern

Java 19 allows the record to be deconstructed directly:

```
if (obj instanceof Person(String name, int age)) {  
    System.out.println(name);  
    System.out.println(age);  
}
```

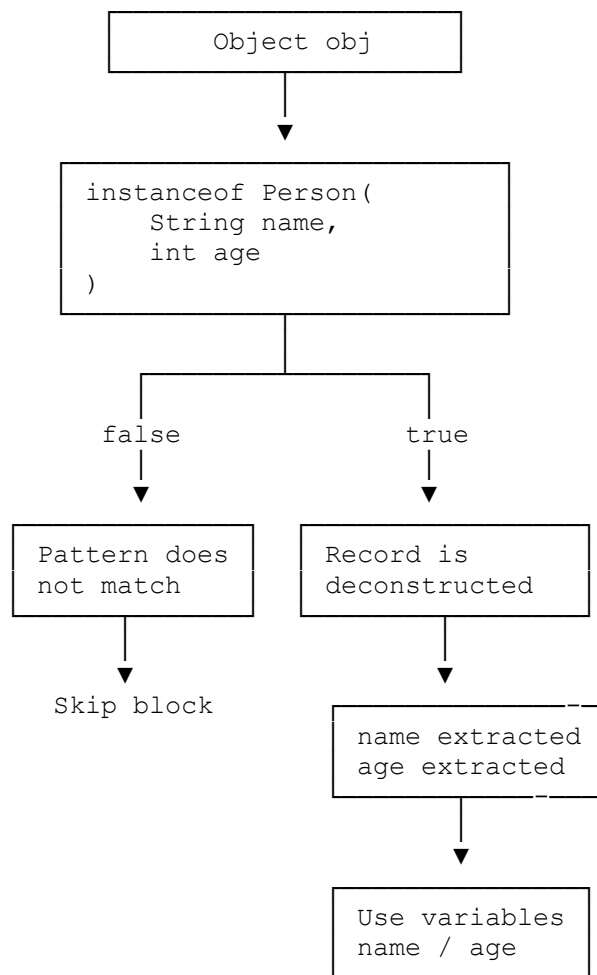
The pattern:

`Person(String name, int age)`

matches a Person record and extracts its components into name and age.

---

## Flowchart



## Java Code Example

Because Record Patterns were a **preview feature in Java 19**, preview support must be enabled.

```
public class RecordPatternsPreview {  
    record Person(String name, int age) {  
    }  
  
    public static void main(String[] args) {  
        Object obj = new Person("Anurag", 30);  
  
        if (obj instanceof Person(String name, int age)) {  
            System.out.println("Name: " + name);  
            System.out.println("Age : " + age);  
        }  
    }  
}
```

## Compile

```
javac --enable-preview --release 19 RecordPatternsPreview.java
```

## Run

```
java --enable-preview RecordPatternsPreview
```

## Output

```
Name: Anurag  
Age : 30
```

---

## Traditional Approach vs Record Pattern

### Traditional approach

```
record Person(String name, int age) {}  
  
Object obj = new Person("Anurag", 30);  
  
if (obj instanceof Person person) {  
    String name = person.name();  
    int age = person.age();  
  
    System.out.println(name);  
    System.out.println(age);  
}
```

```
}
```

## Record Pattern

```
record Person(String name, int age) {}

Object obj = new Person("Anurag", 30);

if (obj instanceof Person(String name, int age)) {

    System.out.println(name);
    System.out.println(age);
}
```

The record pattern eliminates the intermediate person variable and the explicit accessor calls.

---

## Nested Record Patterns

Record patterns can also be **nested**.

Consider:

```
record Address(String city, String country) {}

record Person(String name, Address address) {}
```

We can deconstruct both records in one pattern:

```
public class RecordPatternsPreview {

    record Address(String city, String country) {
    }

    record Person(String name, Address address) {
    }

    public static void main(String[] args) {

        Person person = new Person(
            "Anurag",
            new Address("Bilaspur", "India")
        );

        if (person instanceof Person(
            String name,
            Address(String city, String country)
        )) {

            System.out.println("Name      : " + name);
            System.out.println("City      : " + city);
        }
    }
}
```

```

        System.out.println("Country : " + country);
    }
}

```

## Output

Name : Anurag  
 City : Bilaspur  
 Country : India

The structure can be visualized as:

```

Person
├─ name
├─ Address
│   └─ city
│   └─ country

```

The record pattern directly follows that structure:

```

Person(
    String name,
    Address(
        String city,
        String country
    )
)

```

## Record Pattern with Conditions

Record patterns can be combined with conditions:

```

public class RecordPatternsPreview {

    record Person(String name, int age) {

    }

    public static void main(String[] args) {

        Object obj = new Person("Anurag", 30);

        if (obj instanceof Person(String name, int age)
            && age >= 18) {

            System.out.println(
                name + " is an adult."
            );
        }
    }
}

```

```
}
```

Output:

Anurag is an adult.

---

### Key Characteristics

| Characteristic          | Record Patterns     |
|-------------------------|---------------------|
| Introduced              | Java 19             |
| Year                    | 2022                |
| Status                  | Preview             |
| Purpose                 | Deconstruct records |
| Works with              | Records             |
| Works with instanceof   | Yes                 |
| Nested patterns         | Yes                 |
| Explicit accessor calls | Reduced             |
| Final feature           | Java 21             |

### In short

**Record Patterns in Java 19 provide a concise way to deconstruct records during pattern matching, allowing record components to be extracted directly into variables.** They were introduced as a preview in Java 19 and became a **final feature in Java 21**.